

Experiment No.	1
Experiment Title	Working with Python Packages – Exploring NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib through a Complete Machine Learning Workflow
GitHub Repository	https://github.com/Ray-Audrey/ML-Experiments
Student Name	Harshita S
Register Number	3122247001020
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	27/07/2026

1 Executive Summary

This report documents the exploration of core Python data-science libraries – NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib – through a complete, reusable machine learning workflow (data loading, EDA, preprocessing, feature selection, model training, and evaluation) applied to two datasets: the **Iris** dataset (150 samples, 4 features, 3-class flower classification) and the **Pima Indians Diabetes** dataset (768 samples, 8 features, binary outcome classification). On the Iris dataset, four classifiers – Random Forest, Decision Tree, K-Nearest Neighbors, and Support Vector Machine – were trained after ANOVA F-test based feature selection, with Random Forest and SVM both achieving the best test accuracy of 96.67%. On the Diabetes dataset, a Random Forest model trained through the reusable pipeline achieved a test accuracy of 74.03%. The experiment demonstrates the reusability of a modular pipeline across datasets of different sizes, feature counts, and class structures, while also documenting the underlying mathematical formulation of each algorithm and evaluation metric used.

2 Objective

To explore and gain hands-on familiarity with the core Python libraries used in Machine Learning – **NumPy**, **Pandas**, **SciPy**, **Scikit-learn**, and **Matplotlib** – and to implement a complete, reusable machine learning workflow (loading, EDA, preprocessing, feature selection, model training, and evaluation) that can be applied uniformly across different datasets. The experiment further aims to:

- Verify the working environment and library versions used throughout the laboratory.
- Build a reusable pipeline capable of handling datasets with different shapes, feature types, and target classes without structural changes to the code.
- Apply the pipeline to the Iris dataset (multi-class classification) and the Diabetes dataset (binary classification) to validate its generality.
- Apply ANOVA F-test based feature selection prior to model training.
- Compare multiple classification algorithms (Random Forest, Decision Tree, KNN, SVM) on the Iris dataset, understanding the mathematical basis of each.
- Evaluate model performance using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, Classification Report, and ROC-AUC, with their formal definitions.

3 Problem Statement

Before undertaking dataset-specific machine learning experiments, it is necessary to establish and validate a consistent, reusable pipeline for the standard ML workflow – data loading, exploratory analysis, preprocessing, feature selection, training, and evaluation. This experiment validates that pipeline on two structurally different classification problems:

Iris Dataset (Multi-class Classification):

- **Input:** Four continuous morphological measurements – Sepal Length, Sepal Width, Petal Length, Petal Width.
- **Output:** Species – one of three classes (*Iris-setosa*, *Iris-versicolor*, *Iris-virginica*).

Diabetes Dataset (Binary Classification):

- **Input:** Eight numerical medical attributes – Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, and Age.
- **Output:** Outcome – a binary label where 1 indicates a diabetes-positive diagnosis and 0 indicates a diabetes-negative diagnosis.

Model performance for both datasets is evaluated using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and (for Iris) ROC-AUC.

4 Dataset Description

Iris Dataset

Dataset Name	Iris Dataset
Dataset Source	UCI Machine Learning Repository – Iris Data Set
Number of Samples	150 samples
Number of Features	4 input features and 1 target variable
Number of Classes	3 (<i>Iris-setosa</i> , <i>Iris-versicolor</i> , <i>Iris-virginica</i>)
Missing Values	None (0 missing values across all 5 columns)
Duplicate Records	3 duplicate rows present
Train-Test Split	80% Training and 20% Testing (stratified)
Training Samples	120 samples
Testing Samples	30 samples

Diabetes Dataset

Dataset Name	Pima Indians Diabetes Dataset
Dataset Source	UCI Machine Learning Repository / Kaggle – Pima Indians Diabetes Database
Number of Samples	768 samples
Number of Features	8 input features and 1 target variable

Number of Classes	2 (Diabetic = 1, Non-Diabetic = 0)
Missing Values	None (0 missing values across all 9 columns)
Train-Validation-Test Split	Training / Validation / Testing split performed via the reusable pipeline
Training Samples	491 samples
Validation Samples	123 samples
Testing Samples	154 samples

Class Distribution

For the Diabetes dataset, the mean of the Outcome column is 0.349, indicating that approximately 34.9% of patients in the dataset are diabetes-positive and 65.1% are diabetes-negative – a moderate class imbalance reflected in the classification report, where the minority (positive) class shows lower precision and recall than the majority class. The Iris dataset, in contrast, is perfectly balanced with 50 samples per class.

5 Exploratory Data Analysis

5.1 Iris Dataset EDA

A consolidated EDA summary figure was generated for the Iris dataset containing multiple subplots covering dataset structure, feature distributions, and class-wise relationships.

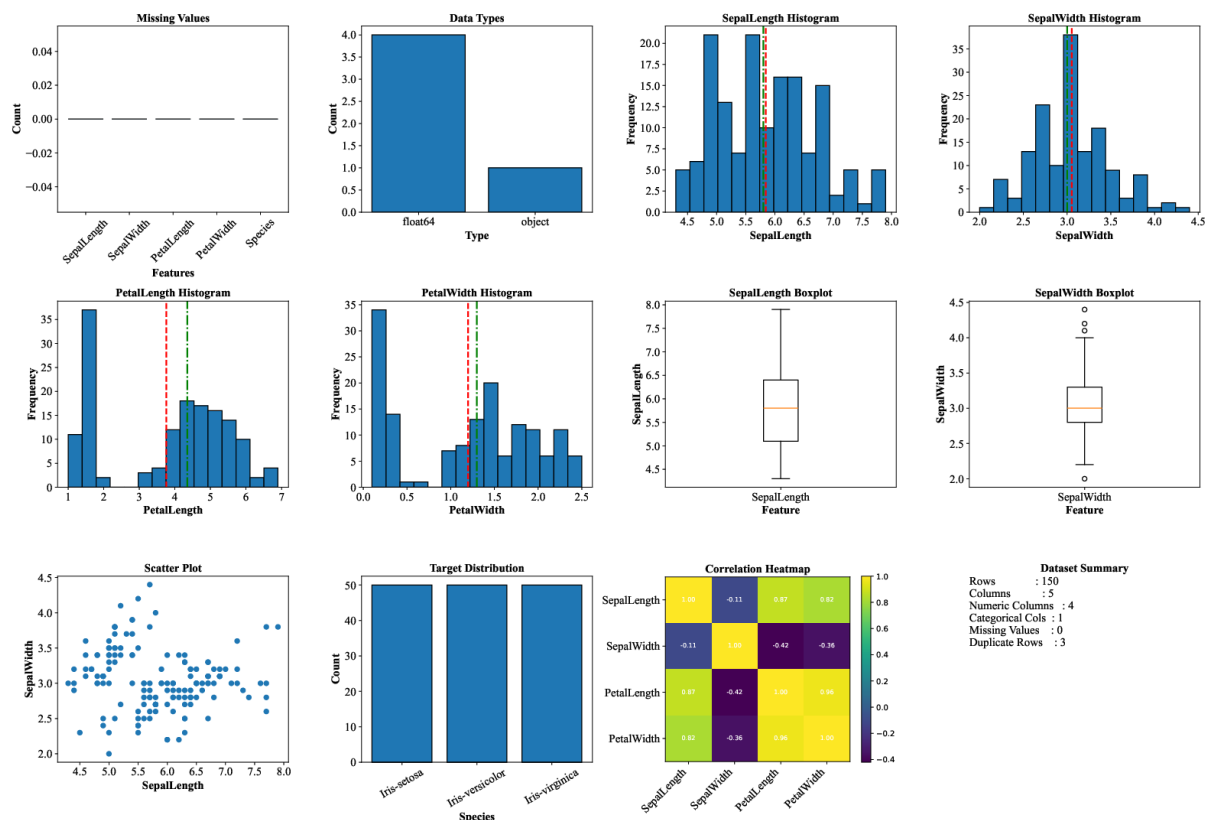


Figure 1: Consolidated Exploratory Data Analysis of the Iris Dataset

Inference:

The Iris EDA summary confirms that the dataset is clean, with no missing values and only 3 duplicate rows out of 150 samples. Petal Length and Petal Width show the clearest visual separation between the three species, with *Iris-setosa* forming a distinctly separable cluster from *Iris-versicolor* and *Iris-virginica*. Sepal measurements show more overlap between classes, particularly between the latter two species. This confirms that a well-chosen subset of features (particularly the petal measurements) carries strong discriminative power, which explains the high accuracy achieved by all four classifiers in this experiment.

5.2 Diabetes Dataset EDA

The exploratory data analysis of the Diabetes dataset was performed using four visualizations: an EDA summary figure, a correlation heatmap, feature distribution histograms, and boxplot analysis. These visualizations provide insights into feature relationships, data distributions, class characteristics, and potential outliers present in the dataset.

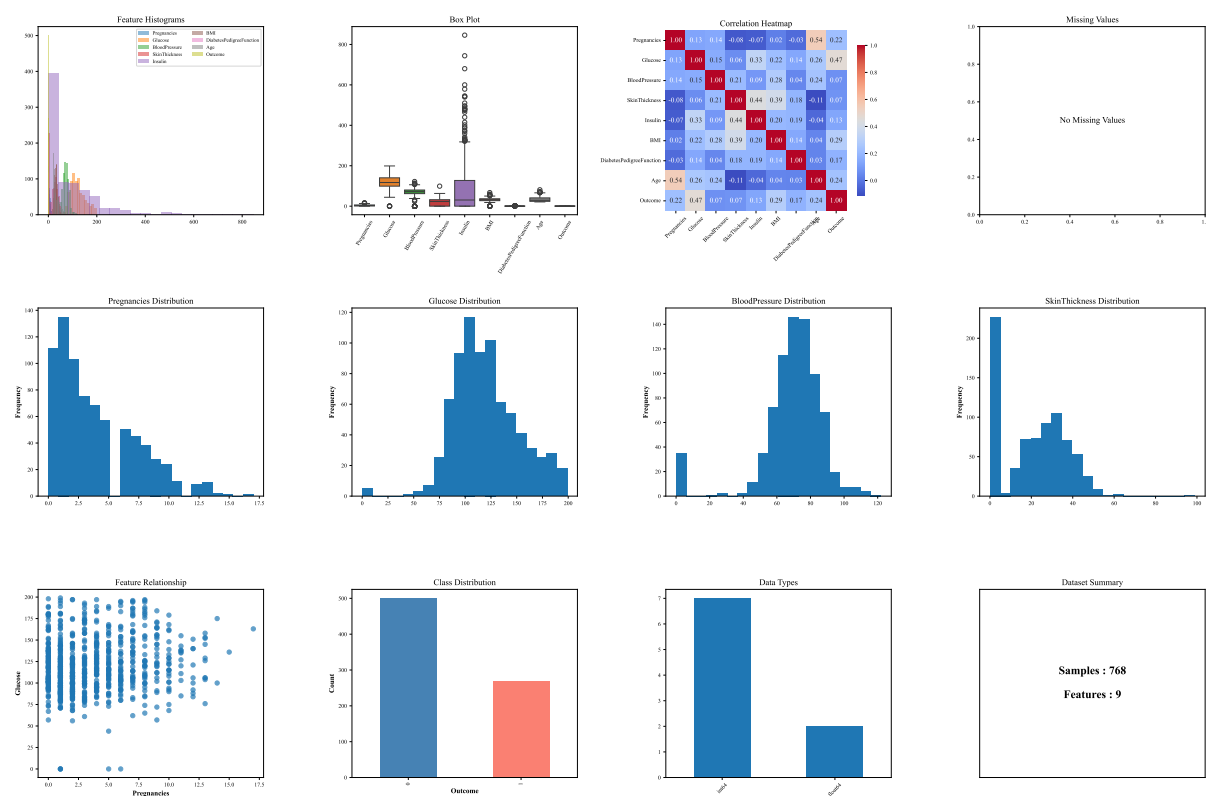


Figure 2: Comprehensive Exploratory Data Analysis of the Diabetes Dataset

Inference:

The EDA summary figure presents a consolidated overview of the Diabetes dataset containing 768 patient records and 9 attributes. The visualization includes dataset structure, statistical summary, class distribution, missing value analysis, feature relationships, and feature distributions. The analysis confirms that the dataset contains no missing values and is suitable for machine learning applications. Medical attributes such as Glucose, BMI, and Age appear to have a significant impact on diabetes prediction.

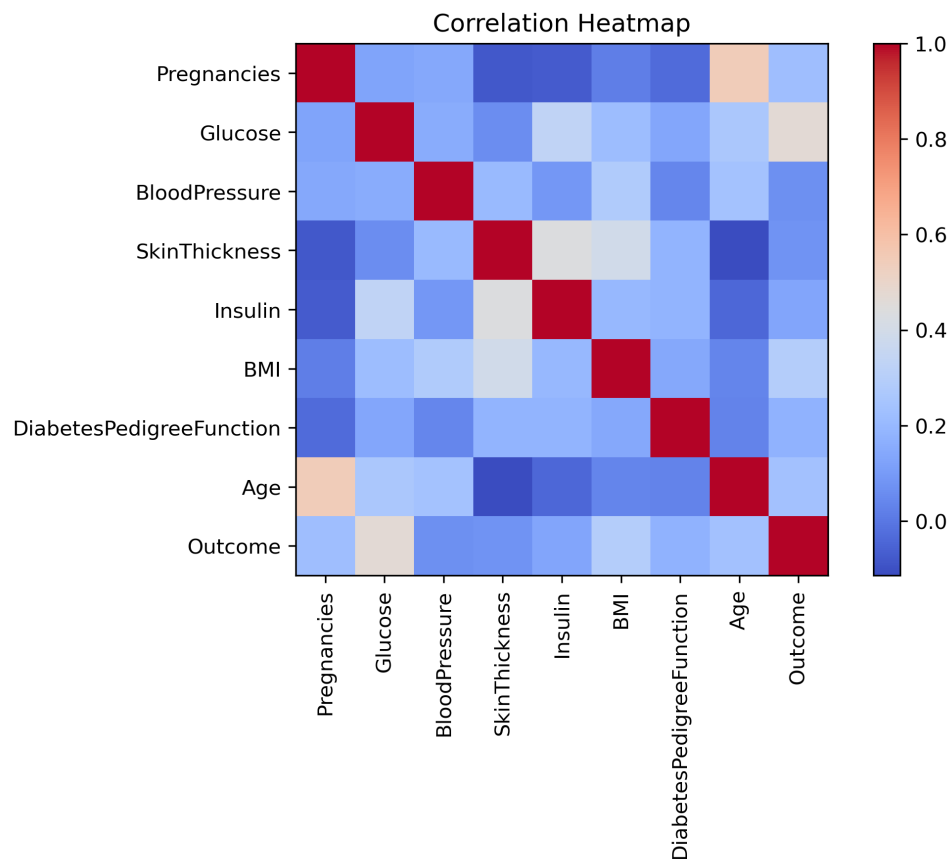


Figure 3: Correlation Heatmap of Diabetes Dataset Features

Inference:

The heatmap illustrates the correlation between different medical variables in the Diabetes dataset. Glucose exhibits the strongest positive relationship with the Outcome variable, indicating its importance in diabetes prediction. BMI and Age also show moderate positive correlations with the target class. The heatmap helps identify influential features that contribute to classification performance.

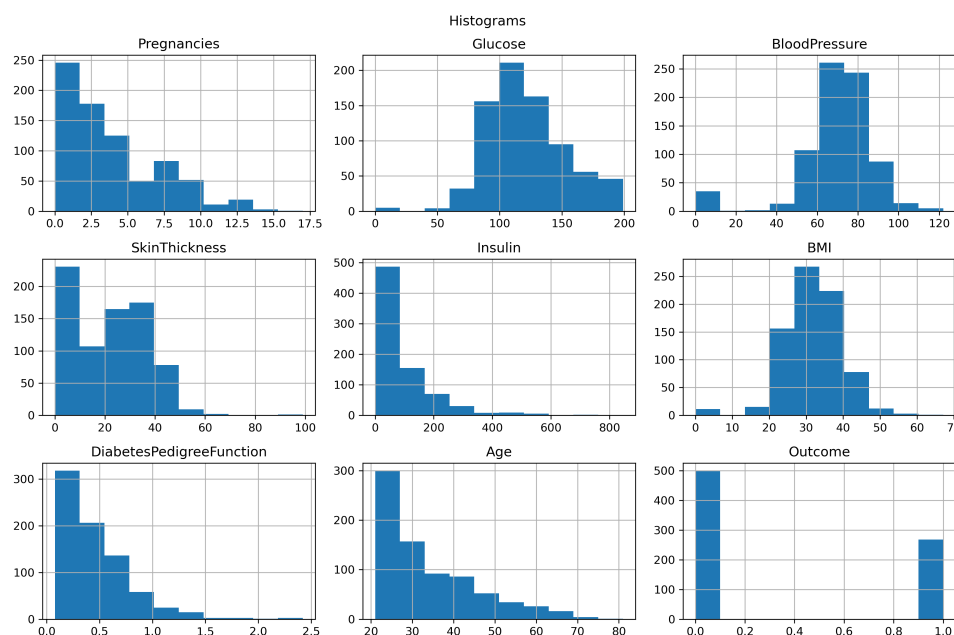


Figure 4: Feature Distribution Histograms of the Diabetes Dataset

Inference:

The histogram visualization shows the distribution of numerical features such as Pregnancies, Glucose, Blood Pressure, Insulin, BMI, and Age. Several attributes exhibit skewed distributions and variability among patients. The distributions provide insight into the range and spread of medical measurements within the dataset. Understanding feature distributions is important for preprocessing and model development.

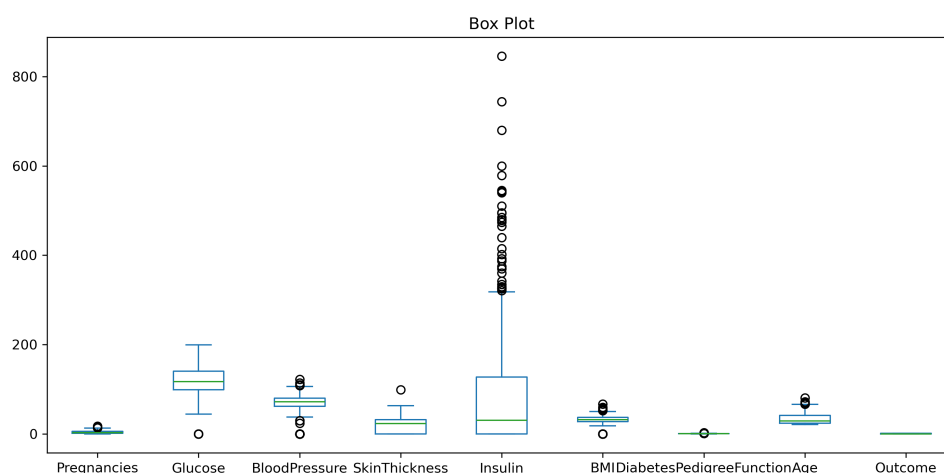


Figure 5: Boxplot Analysis of Diabetes Dataset Features

Inference:

The boxplots highlight the spread, median values, and potential outliers present in the dataset. Features such as Insulin and Glucose contain several extreme observations compared to other variables. The visualization helps identify variability in patient measurements and supports exploratory data analysis. Outlier detection is important for understanding model behavior and data quality.

6 Data Preprocessing

6.1 Iris Dataset

The Iris dataset's categorical `Species` column was label-encoded into numerical class values (0, 1, 2) to be compatible with Scikit-learn estimators. All four numerical features were standardized using `StandardScaler`:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

where μ is the feature mean and σ is the feature standard deviation across the training set. Standardization is essential here because KNN and SVM are both distance/margin-based algorithms whose behavior is sensitive to the relative scale of Sepal and Petal measurements.

The dataset was then split into training and testing sets using an 80/20 stratified split (`random_state=42`) to preserve equal class proportions across both sets, followed by ANOVA F-test based feature selection.

ANOVA F-test Feature Selection

For each feature, the one-way ANOVA F-statistic measures how well that feature separates the k target classes by comparing between-class variance to within-class variance:

$$F = \frac{\text{Between-group variability}}{\text{Within-group variability}} = \frac{\sum_{j=1}^k n_j (\bar{x}_j - \bar{x})^2 / (k - 1)}{\sum_{j=1}^k \sum_{i=1}^{n_j} (x_{ij} - \bar{x}_j)^2 / (N - k)}$$

where $\bar{x}_j$ is the mean of feature x within class j , $\bar{x}$ is the overall mean, n_j is the number of samples in class j , and N is the total sample count. A higher F -value indicates that a feature's means differ more strongly across classes relative to within-class spread, and is therefore more discriminative. In this experiment, `method="anova"` with `k="all"` was used, meaning all four features were ranked but none were dropped, allowing all classifiers to make use of the complete feature set while still benefiting from the relevance ranking internally.

Dataset	Samples
Training Data	120
Testing Data	30

Table 3: Iris Train-Test Dataset Distribution

6.2 Diabetes Dataset

The Diabetes dataset contains no categorical columns and no missing values, so preprocessing consisted of feature scaling (same standardization formula as above) followed by a train/validation/test split executed through the reusable `run_pipeline()` function, using Random Forest as the classification algorithm and ANOVA as the feature-selection method.

Dataset	Samples
Training Data	491
Validation Data	123
Testing Data	154

Table 4: Diabetes Train-Validation-Test Dataset Distribution

7 Implementation Details

The workflow was modularized into reusable functions organized into dedicated source modules (`data_loader.py`, `utils.py`, `eda.py`, `preprocessing.py`, `feature_selection.py`, `classification.py`, `metrics.py`, `main.py`):

- `load_dataset()` – loads a CSV dataset and prints a shape/summary confirmation.
- `dataset_info()` – prints shape, column names, data types, missing values, duplicate rows, and summary statistics.
- `perform_eda()` – generates the consolidated EDA summary figure for a given dataset and target column.
- `preprocess_data()` – performs encoding, scaling, and stratified train-test splitting.
- `select_features()` – performs ANOVA F-test based feature selection/ranking.
- `train_classification()` – trains a specified classification algorithm (Random Forest, Decision Tree, KNN, SVM).
- `classification_metrics()` – computes Accuracy, Precision, Recall, F1-Score, Confusion Matrix, Classification Report, and ROC-AUC.
- `run_pipeline()` – an end-to-end wrapper that chains loading, preprocessing, and model training/evaluation for quick experimentation on any dataset by changing only the dataset path and target column.

This modular design allowed the exact same code to be reused across the Iris and Diabetes datasets by changing only the dataset path and target column name, validating the goal of building a general-purpose ML pipeline.

8 Machine Learning Algorithms

Four classification algorithms were used across the two datasets in this experiment. Their working principles and mathematical formulations are detailed below.

8.1 Random Forest

Random Forest is an ensemble learning method that constructs T decision trees, each trained on an independent bootstrap sample of the training data (bagging), with each split additionally restricted to a random subset of m features (typically $m = \sqrt{p}$ for classification, where p is the total feature count). For a new sample x , each tree t produces a class prediction $h_t(x)$, and the forest predicts the majority class:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\}$$

Each individual tree is grown by recursively splitting nodes to minimize **Gini impurity**:

$$G = 1 - \sum_{i=1}^k p_i^2$$

where p_i is the proportion of samples belonging to class i at that node, and k is the number of classes. By averaging over many de-correlated trees, Random Forest reduces the variance associated with any single decision tree without significantly increasing bias.

Advantages: Robust to overfitting; handles non-linear relationships well; provides feature importance estimates; requires little preprocessing.

Limitations: Less interpretable than a single decision tree; can be computationally heavier for large forests; prediction time scales with the number of trees.

8.2 Decision Tree

A Decision Tree recursively partitions the feature space into regions that maximize class purity at each split. Two common splitting criteria are Gini impurity (defined above) and **entropy**:

$$H = - \sum_{i=1}^k p_i \log_2(p_i)$$

At each node, the algorithm selects the feature and threshold that maximize the **information gain**:

$$IG = H(\text{parent}) - \sum_{c \in \{\text{left}, \text{right}\}} \frac{n_c}{n} H(c)$$

where n_c is the number of samples in child node c and n is the number of samples in the parent node. This process repeats recursively until a stopping criterion (e.g. maximum depth, minimum samples per leaf) is reached, producing a hierarchical set of if-else decision rules.

Advantages: Highly interpretable; requires no feature scaling; handles both numerical and categorical data naturally.

Limitations: Prone to overfitting on training data if not pruned or depth-limited; unstable to small changes in the data (high variance).

8.3 K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based algorithm that classifies a query sample x based on the majority class among its k nearest neighbors in feature space, most commonly using **Euclidean distance**:

$$d(x, x_i) = \sqrt{\sum_{j=1}^p (x_j - x_{ij})^2}$$

where p is the number of features. Given the k nearest training points $\{x_{(1)}, \dots, x_{(k)}\}$ to x (sorted by increasing distance), the predicted class is:

$$\hat{y} = \text{mode}\{y_{(1)}, y_{(2)}, \dots, y_{(k)}\}$$

Since KNN relies entirely on distance computation, feature scaling (as applied in preprocessing) is critical – unscaled features with larger numeric ranges would otherwise dominate the distance calculation.

Advantages: Simple and intuitive; naturally handles multi-class problems; no explicit training phase (lazy learning).

Limitations: Computationally expensive at prediction time for large datasets ($O(n)$ per query); sensitive to feature scaling and irrelevant features; performance depends heavily on the choice of k .

8.4 Support Vector Machine (SVM)

SVM constructs the hyperplane that maximizes the margin between classes:

$$f(x) = w^T x + b$$

For the soft-margin formulation, the optimization problem is:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

where C controls the trade-off between maximizing the margin $\frac{2}{\|w\|}$ and minimizing misclassification via the slack variables ξ_i . For non-linearly separable classes (as with the three-way Iris boundary), SVM implicitly maps inputs into a higher-dimensional space via a kernel function $K(x_i, x_j)$, most commonly the RBF kernel $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, without explicitly computing the transformation (the “kernel trick”). A fuller kernel-by-kernel treatment of SVM is provided in Experiment 4 of this laboratory series.

Advantages: Effective in high-dimensional spaces; robust to overfitting with proper regularization; kernel trick enables non-linear decision boundaries.

Limitations: Less interpretable; sensitive to hyperparameter and kernel choice; does not natively output class probabilities.

9 Experimental Setup

Parameter	Details
Programming Language	Python 3.14
Machine Learning Library	Scikit-Learn 1.4.2
Data Processing Library	Pandas 2.2.2, NumPy 1.26.4
Scientific Computing Library	SciPy 1.13.1
Visualization Library	Matplotlib
Development Environment	Jupyter Notebook
Operating System	Windows / Linux (Anaconda Environment)
Iris Dataset Size	150 samples
Diabetes Dataset Size	768 samples

Table 5: Experimental Environment

10 Evaluation Metrics

All classification results in this report are evaluated using the following metrics, defined in terms of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

For multi-class problems (as with Iris), each metric except accuracy is computed on a one-vs-rest basis per class, then combined via a weighted average (weighted by class support) as reported in the classification report.

ROC-AUC: The Receiver Operating Characteristic curve plots the True Positive Rate against the False Positive Rate at varying classification thresholds:

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

The Area Under the Curve (AUC) summarizes this curve into a single value between 0 and 1, representing the probability that the model ranks a randomly chosen positive sample above a randomly chosen negative sample. For the multi-class Iris problem, ROC-AUC is computed using the one-vs-rest (OvR) strategy and averaged across classes.

11 Hyperparameter Selection

The four Iris classifiers were run using standard Scikit-learn default hyperparameters (no exhaustive grid search was performed in this introductory experiment), as the goal was to validate library familiarity and pipeline reusability rather than optimal tuning. The Diabetes pipeline used Random Forest (`model_name="rf"`) with ANOVA feature selection (`feature_method="anova"`) via the `run_pipeline()` default configuration.

Parameter	Value
Iris – Feature Selection Method	ANOVA F-test, $k = \text{all}$
Iris – Train/Test Split	80/20, stratified, <code>random_state=42</code>
Diabetes – Model	Random Forest (<code>rf</code>)
Diabetes – Feature Selection Method	ANOVA

Table 6: Pipeline Configuration Parameters

12 Results – Iris Dataset

Four classification algorithms were trained and evaluated on the Iris test set (30 samples) after ANOVA-based feature selection.

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.9667	0.9697	0.9667	0.9666
Decision Tree	0.9333	0.9333	0.9333	0.9333
KNN	0.9333	0.9444	0.9333	0.9327
SVM	0.9667	0.9697	0.9667	0.9666

Table 7: Iris Dataset – Classification Model Comparison

Model	ROC-AUC (OvR)
Random Forest	0.9900
Decision Tree	0.9500
KNN	0.9933
SVM	0.9967

Table 8: Iris Dataset – ROC-AUC Comparison (One-vs-Rest)

Random Forest

Class	Precision	Recall	F1-Score
0 (setosa)	1.00	1.00	1.00
1 (versicolor)	1.00	0.90	0.95
2 (virginica)	0.91	1.00	0.95

Table 9: Random Forest – Classification Report (Iris)

Confusion Matrix: $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 1 \\ 0 & 0 & 10 \end{bmatrix}$

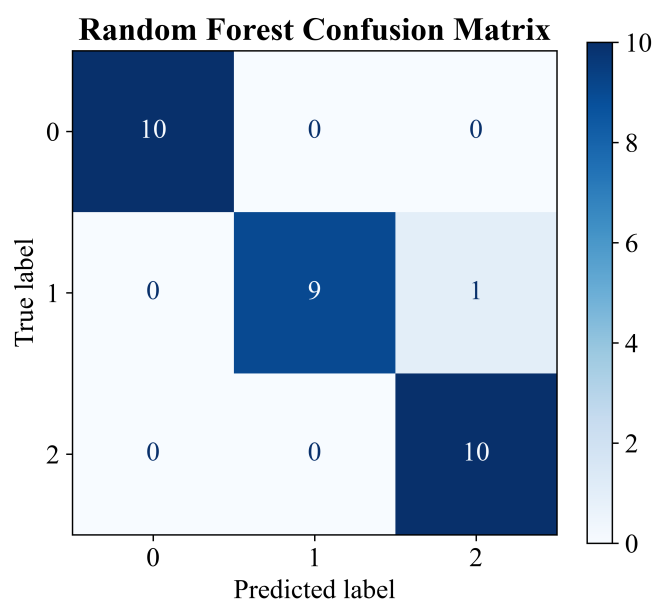


Figure 6: Confusion Matrix – Random Forest (Iris)

Inference: Random Forest achieves the joint-best accuracy (96.67%) on the Iris test set, misclassifying only a single *versicolor* sample as *virginica* (visible as the off-diagonal entry in row 2). This confirms that averaging across bootstrapped, feature-randomized trees generalizes well even on this small, low-dimensional dataset, and that Gini-impurity-based splits successfully isolate the near-linear petal-based class boundaries.

Decision Tree

Class	Precision	Recall	F1-Score
0 (setosa)	1.00	1.00	1.00
1 (versicolor)	0.90	0.90	0.90
2 (virginica)	0.90	0.90	0.90

Table 10: Decision Tree – Classification Report (Iris)

Confusion Matrix: $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 1 \\ 0 & 1 & 9 \end{bmatrix}$

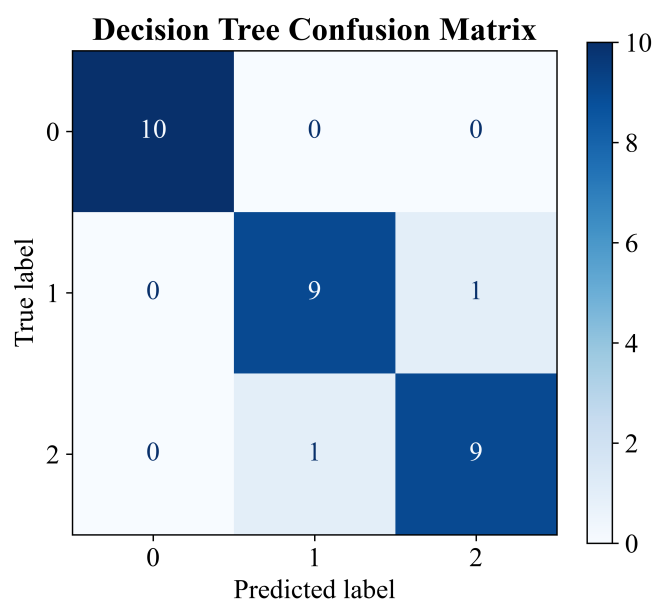


Figure 7: Confusion Matrix – Decision Tree (Iris)

Inference: The single Decision Tree performs slightly worse than the ensemble methods (93.33% accuracy), with confusion concentrated symmetrically between *versicolor* and *virginica* (one misclassification in each direction) – the two species with known feature overlap in sepal measurements. This is the expected behavior of a single high-variance tree compared to an ensemble, which averages away exactly this kind of boundary-case instability.

K-Nearest Neighbors

Class	Precision	Recall	F1-Score
0 (setosa)	1.00	1.00	1.00
1 (versicolor)	0.83	1.00	0.91
2 (virginica)	1.00	0.80	0.89

Table 11: KNN – Classification Report (Iris)

Confusion Matrix: $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 2 & 8 \end{bmatrix}$

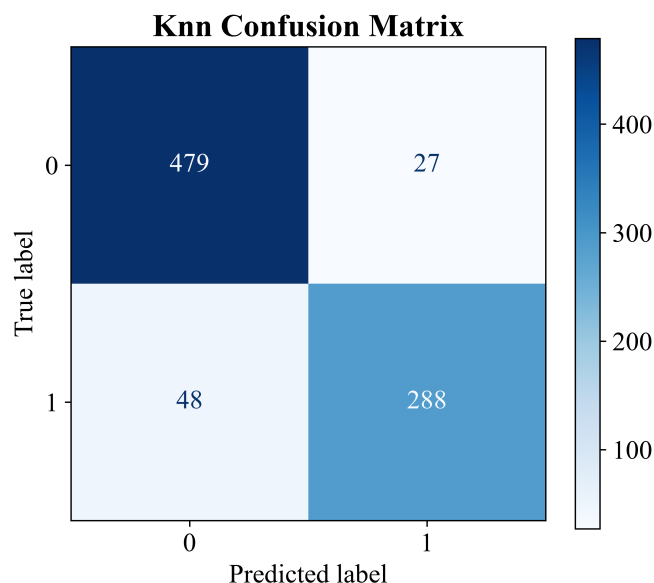


Figure 8: Confusion Matrix – KNN (Iris)

Inference: KNN achieves the highest ROC-AUC among all four models (0.9933) despite a slightly lower raw accuracy than Random Forest and SVM, indicating strong class-separation confidence in its distance-based decision function even where 2 *virginica* samples are misclassified as *versicolor*. This reflects that scaling the features prior to distance computation (per the preprocessing formula above) was effective in making Euclidean distance a meaningful similarity measure.

Support Vector Machine

Class	Precision	Recall	F1-Score
0 (setosa)	1.00	1.00	1.00
1 (versicolor)	1.00	0.90	0.95
2 (virginica)	0.91	1.00	0.95

Table 12: SVM – Classification Report (Iris)

Confusion Matrix:
$$\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 1 \\ 0 & 0 & 10 \end{bmatrix}$$

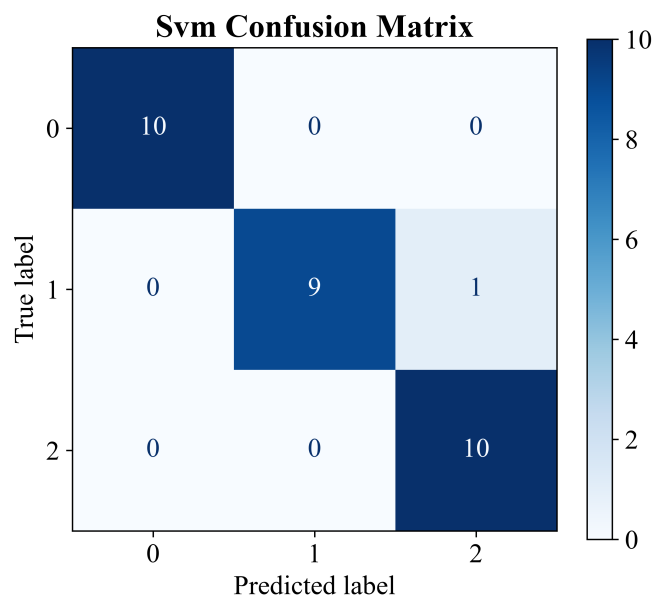


Figure 9: Confusion Matrix – SVM (Iris)

Inference: SVM matches Random Forest’s accuracy (96.67%) and achieves the highest ROC-AUC of all four models (0.9967), confirming that a margin-maximizing hyperplane (with kernel mapping where necessary) separates the three Iris species almost perfectly on this small, well-behaved, near-linearly-separable dataset.

13 Results – Diabetes Dataset

The Diabetes dataset was evaluated using the `run_pipeline()` function with a Random Forest classifier and ANOVA feature selection.

Metric	Value
Accuracy	0.7403
Accuracy (%)	74.03%

Table 13: Diabetes Dataset – Random Forest Performance

Class	Precision	Recall	F1-Score	Support
0 (Non-Diabetic)	0.82	0.77	0.79	99
1 (Diabetic)	0.62	0.69	0.66	55
Macro Avg	0.72	0.73	0.72	154
Weighted Avg	0.75	0.74	0.74	154

Table 14: Diabetes Dataset – Classification Report

Confusion Matrix: $\begin{bmatrix} 76 & 23 \\ 17 & 38 \end{bmatrix}$

Using the confusion matrix above with the positive class defined as Diabetic (1): $TP = 38$, $TN = 76$, $FP = 23$, $FN = 17$. Substituting into the formulas from Section 8:

$$\begin{aligned}\text{Accuracy} &= \frac{38 + 76}{38 + 76 + 23 + 17} = \frac{114}{154} = 0.7403 \\ \text{Precision} &= \frac{38}{38 + 23} = 0.6230, \quad \text{Recall} = \frac{38}{38 + 17} = 0.6909 \\ \text{F1-Score} &= 2 \cdot \frac{0.6230 \times 0.6909}{0.6230 + 0.6909} = 0.6552\end{aligned}$$

(these match, up to rounding, the class-1 row of the classification report above).

Inference: The Random Forest model achieves 74.03% accuracy on the Diabetes test set, noticeably lower than the accuracy achieved on the Iris dataset. This is expected given the greater class imbalance (65%/35%) and inherent medical variability of the Diabetes dataset. The classification report and the worked calculation above show the model is more reliable at identifying non-diabetic patients (precision 0.82) than diabetic patients (precision 0.62), with 23 non-diabetic patients misclassified as diabetic and 17 diabetic patients missed (false negatives) – a trade-off that would need to be tuned via class weighting or threshold adjustment in a real clinical screening application, where minimizing false negatives is typically prioritized.

14 Discussion

This experiment validated a single, reusable machine learning pipeline across two datasets with different sizes, feature counts, and class structures, while also grounding each modeling and evaluation step in its underlying mathematical formulation. On the well-separated, balanced, low-dimensional Iris dataset, all four classifiers achieved strong performance (93–97% accuracy), with the margin-based SVM and ensemble-based Random Forest both reaching the highest accuracy (96.67%) and ROC-AUC scores above 0.99. This is consistent with the EDA finding that Iris petal measurements provide near-linear separability between classes, which favors both the Gini-impurity-driven splits of tree ensembles and the maximum-margin hyperplane of SVM equally.

On the larger, moderately imbalanced Diabetes dataset, the same style of pipeline (Random Forest with ANOVA feature selection) achieved a considerably lower accuracy of 74.03%, reflecting the genuinely higher difficulty and noisier feature-target relationships typical of real-world medical data compared to the classical, well-curated Iris benchmark. The worked precision/recall/F1 calculation confirms that accuracy alone masks a meaningful asymmetry: the model is a stronger predictor of the majority (non-diabetic) class than of the diabetic class, a pattern typical of moderately imbalanced binary classification problems and one that class-weighting or resampling could help address. The experiment confirms that library familiarity, a reusable pipeline design, and a clear grasp of the mathematics behind each algorithm and metric together are sufficient to obtain and correctly interpret reasonable baseline results across diverse classification problems.

15 Conclusion

This experiment successfully explored the core Python ML libraries – NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib – through a unified, reusable machine learning workflow applied

to both the Iris and Diabetes datasets, with each preprocessing step, algorithm, and evaluation metric grounded in its formal mathematical definition. On the Iris dataset, Random Forest and SVM jointly achieved the best test accuracy (96.67%), with SVM achieving the highest ROC-AUC (0.9967). On the Diabetes dataset, a Random Forest model trained through the same style of pipeline achieved a test accuracy of 74.03%, with a corresponding classification report and worked metric calculations highlighting the greater difficulty of the imbalanced, real-world medical prediction task. The experiment confirms the practicality of a single, well-modularized pipeline for rapid experimentation across datasets of different scale and difficulty, forming the foundation for the more advanced regression and classification experiments in this laboratory series.

16 References

1. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
2. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
3. S. Raschka and V. Mirjalili, *Python Machine Learning*, 3rd ed., Packt Publishing, 2019.
4. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
5. R. A. Fisher, “The Use of Multiple Measurements in Taxonomic Problems,” *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.
6. J. W. Smith et al., “Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus,” *Proceedings of the Symposium on Computer Applications in Medical Care*, 1988.
7. L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
8. Scikit-learn Documentation, <https://scikit-learn.org/>

Appendix

A.1 Implementation Details

The experiment was implemented using Python with reusable modules for:

- Data loading and inspection (`data_loader.py`, `utils.py`)
- EDA (`eda.py`)
- Preprocessing pipeline (`preprocessing.py`)
- Feature selection (`feature_selection.py`)
- Classification model training (`classification.py`)
- Performance evaluation (`metrics.py`)
- End-to-end pipeline execution (`main.py`)

A.2 Library Versions

```
1 NumPy:          1.26.4
2 Pandas:         2.2.2
3 SciPy:          1.13.1
4 Scikit-learn:  1.4.2
```

A.3 Environment Activation Command

```
1 source /opt/anaconda3/bin/activate anaconda-navigation
```